PRCP-1003 — Customer Transaction Prediction

Final project report

1. Introduction

Banks run targeted offers, reminders and product campaigns. Knowing in advance which customers are likely to make a specific transaction lets the bank contact the right customers, spend its campaign budget efficiently, and avoid bothering customers who are unlikely to respond. This project builds a machine learning model that estimates, for each customer, the probability of making that transaction in the future.

2. Problem Statement

Develop a machine learning model that predicts whether a bank customer will make a specific transaction in the future, irrespective of the amount transacted, using 200 anonymized numeric customer attributes. This is a binary classification problem on imbalanced data.

3. Objectives

- Analyze the customer transaction data
- Identify important patterns
- Perform exploratory data analysis (EDA)
- Preprocess the data
- Train multiple machine learning algorithms
- Compare their performance
- Tune the best model
- Save the final model
- Generate predictions
- Deploy the model

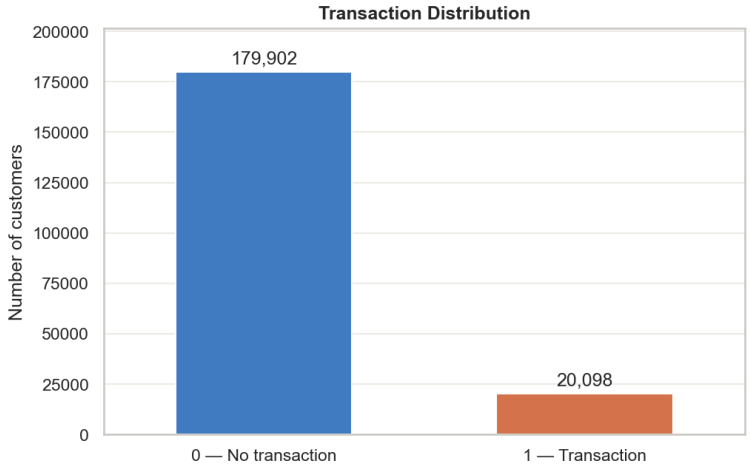
4. Dataset Description

Item	Value
Rows (customers)	200,000
Columns	202 (ID_code, target, 200 features)
Features	var_0 ... var_199: 200 anonymized numeric (float64) customer attributes; names and meaning not disclosed
Identifier	ID_code: text (object), unique per customer, not used as a feature
Target variable	target: int64, 1 = will make the transaction (10.0%), 0 = will not (90.0%)
Data types	200 float64, 1 int64, 1 object
Missing values	0
Duplicate rows	0

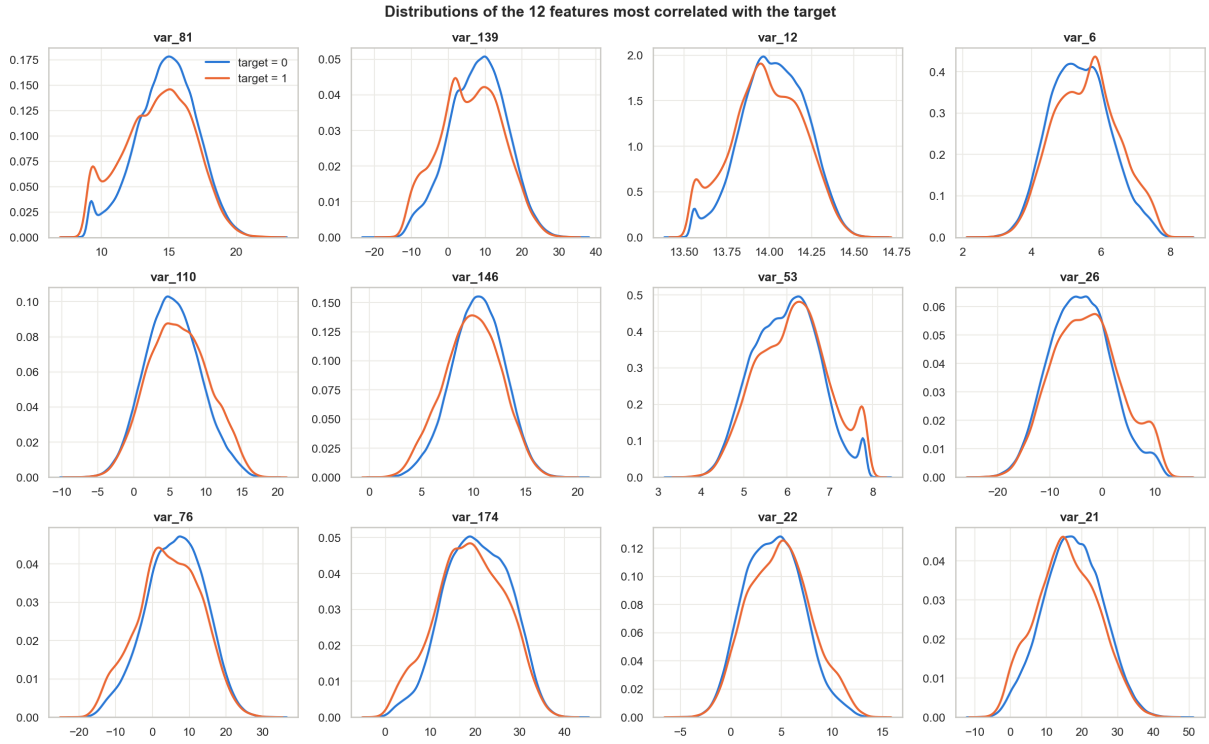
Because the features are anonymized, no business meaning (age, income, balance, ...) can be attached to them. All analysis is statistical.

5. Exploratory Data Analysis

Target distribution. Only 10.0% of customers made the transaction, so the classes are strongly imbalanced. Always predicting 'no transaction' would already be 90.0% accurate.

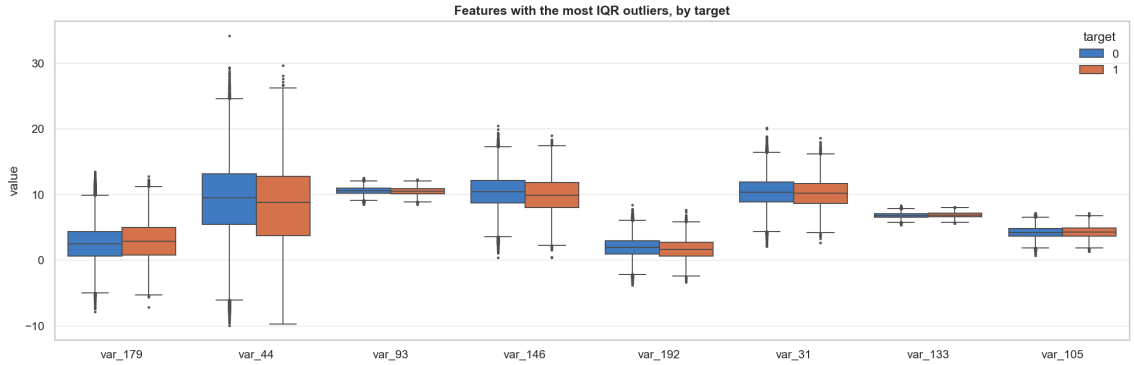


Feature distributions. Almost all features are smooth and roughly bell-shaped. For the most informative features, the two classes show small but consistent shifts; no single feature separates them.

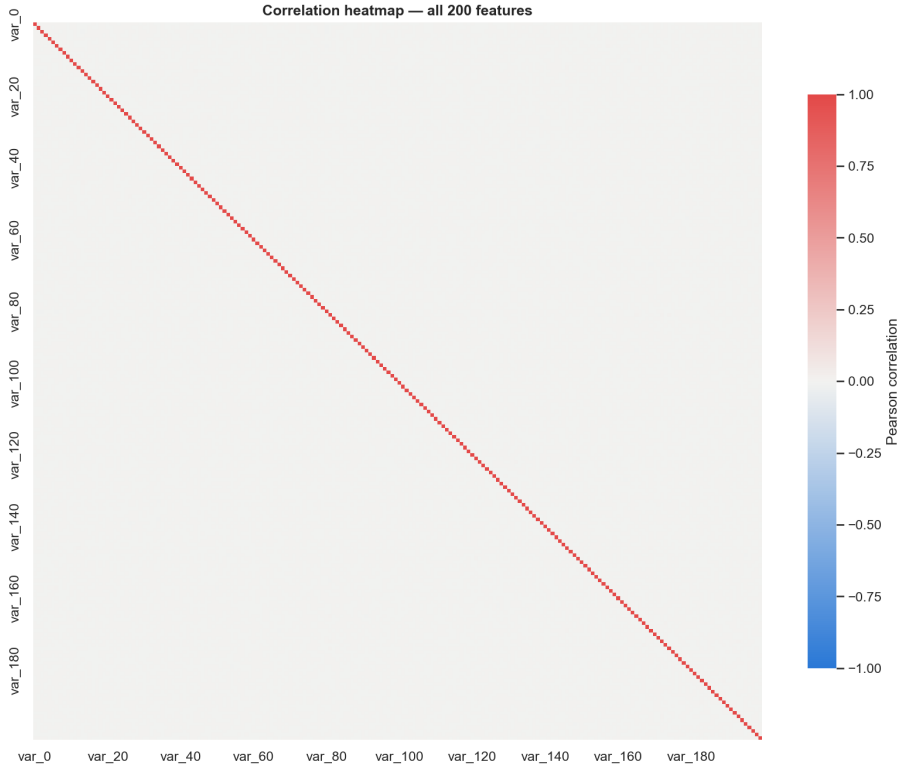


Categorical analysis. The only text column is ID_code, a unique identifier with no predictive meaning. There are no categorical features to encode.

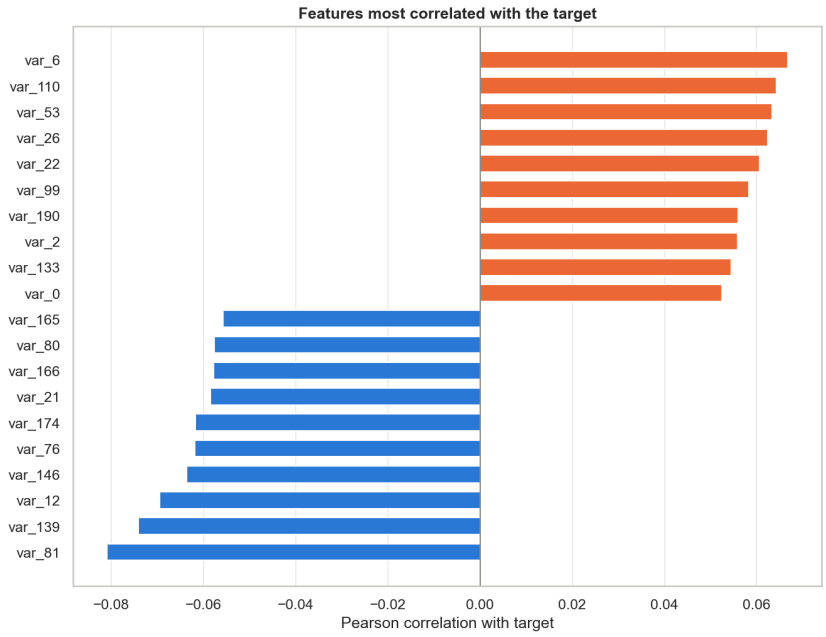
Outlier analysis. 0.07% of values lie outside the IQR fences, fewer than a normal distribution would produce, so they are natural tail values. They appear in 12.4% of rows, and those rows transact more often, so the outliers were kept.



Correlation heatmap. The features are practically uncorrelated with each other (max |r| = 0.010), so there is no multicollinearity.



Important relationships. Each feature is only weakly related to the target (max |r| = 0.081). Strongest positive: var_6, var_110, var_53; strongest negative: var_81, var_139, var_12. Predictive power comes from combining many weak signals.



6. Data Preprocessing

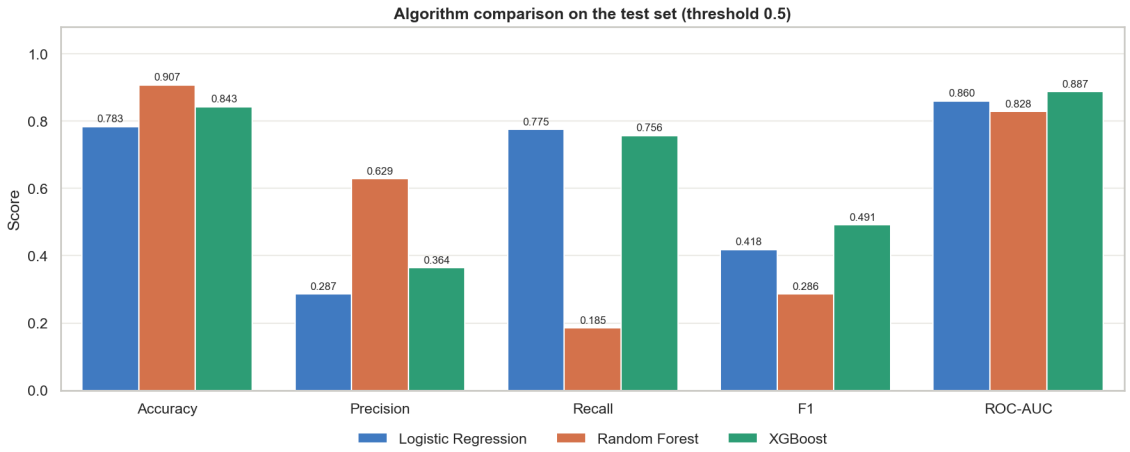
- **Missing-value treatment:** none were present. A SimpleImputer (median / most frequent) is still part of the pipeline to protect the deployed model against incomplete records.
- **Duplicate removal:** drop_duplicates() found 0 duplicate rows.
- **Categorical encoding:** ID_code was excluded as an identifier, leaving no categorical features. A OneHotEncoder branch remains in the pipeline for future categorical columns.
- **Feature scaling:** StandardScaler, because the features have very different ranges; this matters for Logistic Regression.
- **Train/test split:** stratified 80/20 split (160,000 / 40,000 rows), keeping the 10.0% transaction rate in both sets. The scaler is fitted on the training data only (no leakage).

7. Algorithm Selection

- **Logistic Regression:** baseline. Fast, interpretable, outputs probabilities, and suits signals that add up from many independent features.
- **Random Forest:** bagging ensemble of decision trees. Captures non-linear patterns and is robust to outliers.
- **XGBoost:** gradient-boosted trees. Usually the strongest on tabular data, regularised, and handles imbalance through scale_pos_weight.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.7834	0.2865	0.7754	0.4184	0.8599
Random Forest	0.9071	0.6289	0.1851	0.286	0.8279
XGBoost	0.8425	0.3637	0.7565	0.4912	0.8871

XGBoost separated the classes best (highest ROC-AUC and F1). Random Forest had the highest accuracy only because it predicts 'no transaction' almost every time: its recall is very low.



8. Model Training

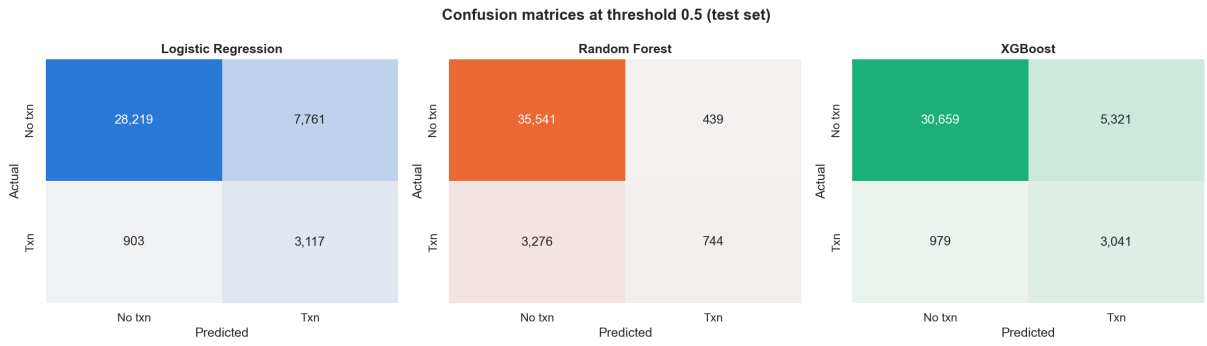
Each algorithm was trained as one scikit-learn Pipeline (ColumnTransformer preprocessing followed by the model) on the training set. Class imbalance was handled with class weights rather than oversampling.

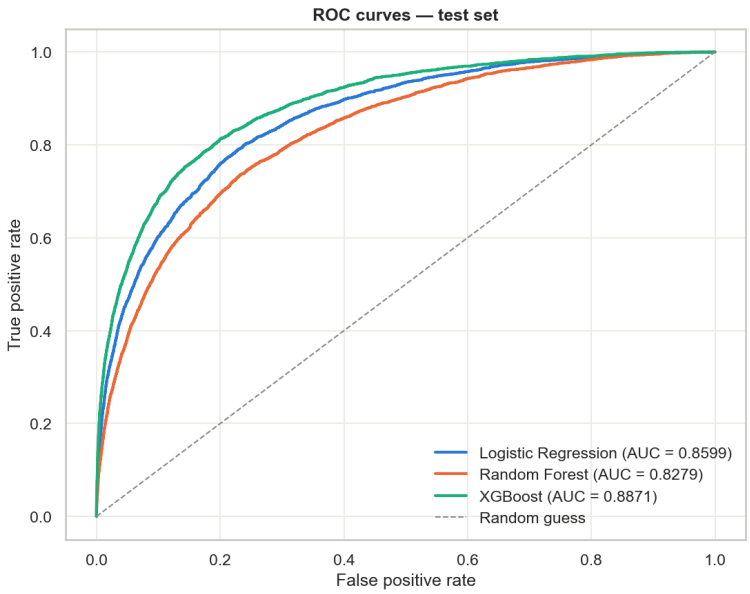
Model	Key settings	Training time (s)
Logistic Regression	max_iter=1000, class_weight=balanced	3.5
Random Forest	200 trees, max_depth=15, min_samples_leaf=20, class_weight=balanced_subsample	60.1
XGBoost	1000 trees, learning_rate=0.05, max_depth=3, scale_pos_weight=8.95	15.2

9. Model Evaluation

- **Accuracy:** share of all predictions that are correct (misleading on imbalanced data).
- **Precision:** when the model says 'transaction', how often it is right.
- **Recall:** share of actual transactions the model finds.
- **F1:** balance of precision and recall.
- **ROC-AUC:** how well the model ranks transacting customers above the rest, across all thresholds.
- **Confusion matrix:** counts of correct and wrong predictions per class.

ROC-AUC was used to choose and tune the algorithm, and F1 to choose the decision threshold.

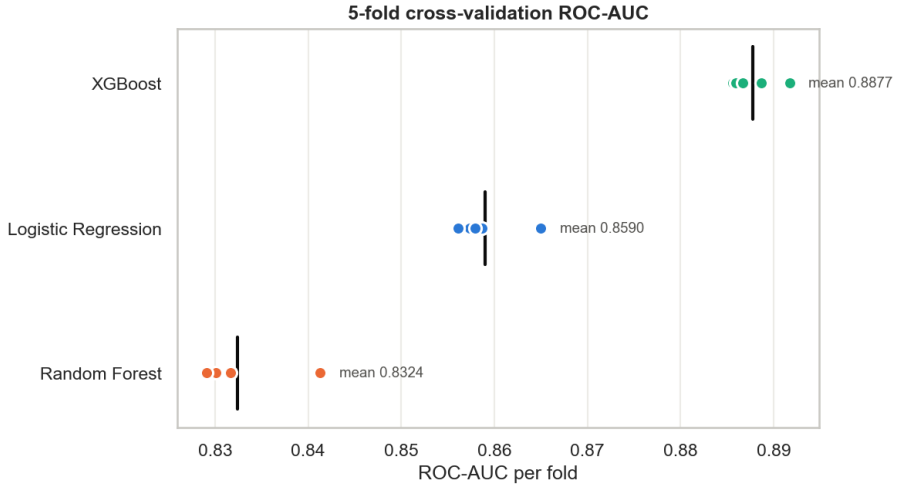




10. Hyperparameter Tuning

Search method. RandomizedSearchCV was used instead of GridSearchCV. The full grid has 9,216 combinations; with 3-fold cross-validation that is 27,648 model fits, which is impractical. Randomized search tested 15 random combinations (45 fits) and usually finds a near-optimal setting at a fraction of the cost.

Cross-validation. 5-fold stratified CV compared the algorithms, and 3-fold stratified CV scored each tuning candidate, always by ROC-AUC on the training set only.



Best parameters: subsample=0.7, reg_lambda=1, n_estimators=2000, min_child_weight=10, max_depth=3, learning_rate=0.05, colsample_bytree=0.3.

Performance before and after tuning (test set):

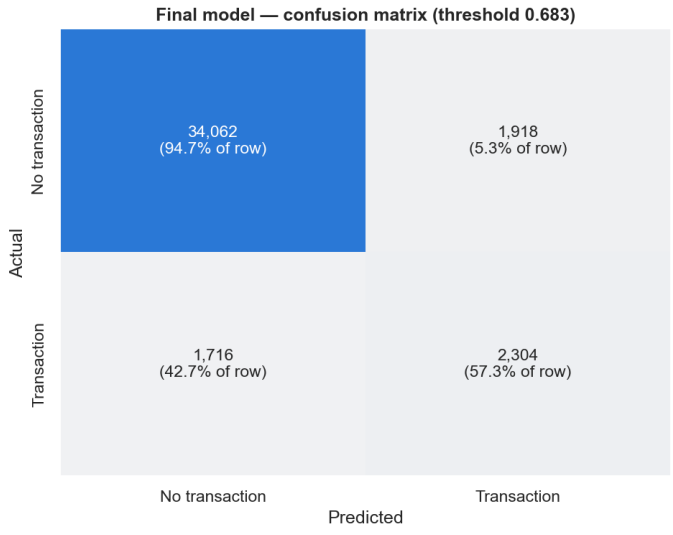
Model	ROC-AUC	CV ROC-AUC	Precision	Recall	F1	Accuracy
Before tuning (threshold 0.5)	0.8871	0.8877	0.3637	0.7565	0.4912	0.8425
After tuning (threshold 0.5)	0.8945	0.8941	0.3854	0.7595	0.5113	0.8541

Model	ROC-AUC	CV ROC-AUC	Precision	Recall	F1	Accuracy
After tuning (threshold 0.683)	0.8945	0.8941	0.5457	0.5731	0.5591	0.9092

11. Final Model

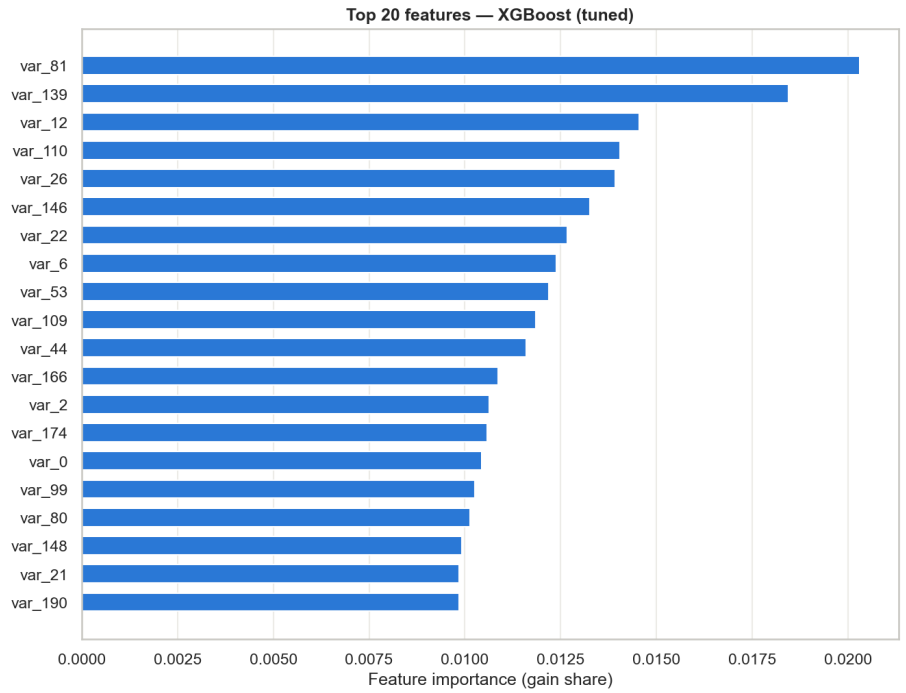
- **Selected algorithm:** XGBoost (tuned), inside a preprocessing pipeline.
- **Final parameters:** subsample=0.7, reg_lambda=1, n_estimators=2000, min_child_weight=10, max_depth=3, learning_rate=0.05, colsample_bytree=0.3; decision threshold 0.683 (maximises F1 on out-of-fold training predictions).
- **Final performance (test set):** ROC-AUC 0.8945, accuracy 0.909, precision 0.546, recall 0.573, F1 0.559.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
XGBoost (tuned)	0.8541	0.3854	0.7595	0.5113	0.8945	0.8941
XGBoost	0.8425	0.3637	0.7565	0.4912	0.8871	0.8877
Logistic Regression	0.7834	0.2865	0.7754	0.4184	0.8599	0.859
Random Forest	0.9071	0.6289	0.1851	0.286	0.8279	0.8324



12. Feature Importance

The most influential attributes are var_81, var_139, var_12, var_110, var_26. Their business meaning is unknown because the bank anonymized them. Importance is spread thinly: the top 20 attributes carry only 25% of the total. The model relies on the combined weak signal of all 200 attributes, which matches the EDA.



13. Prediction

The saved model was reloaded and applied to six test-set customers it never saw during training: three who made the transaction and three who did not. Customers whose probability is at least 0.683 are predicted to transact.

ID_code	Transaction probability	Prediction	Actual
train_85230	0.6918	Transaction	Transaction
train_58973	0.7327	Transaction	Transaction
train_193783	0.3563	No transaction	Transaction
train_70638	0.2353	No transaction	No transaction
train_24697	0.0527	No transaction	No transaction
train_195008	0.0610	No transaction	No transaction

14. Deployment

- **Saved model:** models/customer_transaction_model.pkl holds the full pipeline (preprocessing + XGBoost), the feature list and the decision threshold.
- **FastAPI:** deployment/app.py loads the pipeline once at startup and serves predictions.
- **API endpoints:** GET /health, POST /predict (one customer), POST /predict/batch (many customers). Incomplete input is rejected with HTTP 422.
- **Testing:** tests/test_prediction.py (pytest) checks model loading, input acceptance, prediction and probability output.

15. Limitations

- **Anonymized data:** without feature meanings, the model cannot be explained in business terms, and no domain-driven features could be engineered.

- **Class imbalance:** with only 10.0% positives, precision stays moderate (55% at the chosen threshold), so some contacted customers will not transact.
- **Dataset size and scope:** 200,000 customers from a single snapshot with no time information; behaviour may change over time (data drift).
- **Historical bias:** the model learns from past behaviour and past bank decisions, and will reproduce any bias contained in them.
- **Model generalization:** performance was validated on a random hold-out split, not on later time periods or other customer populations.
- **Threshold choice:** the threshold maximises F1; the real costs of missed vs wasted contacts may call for a different value.

16. Future Improvements

- More customer data, including named attributes and transaction history over time
- Advanced feature engineering (for example value-frequency features, which are known to help on this kind of data)
- Try LightGBM / CatBoost and model ensembles
- Choose the threshold from real campaign costs and benefits
- Model monitoring: track data drift and live precision / recall, retrain on a schedule
- Real-time predictions integrated into the bank's CRM
- A simple web interface for bank staff, with bulk CSV upload and explanations per customer (SHAP)
- Cloud deployment with Docker, authentication and autoscaling

17. Conclusion

The data was clean but strongly imbalanced, with weak signal spread across 200 independent anonymized features. Three algorithms were compared on imbalance-aware metrics. Tuned XGBoost performed best, with test ROC-AUC 0.8945, compared with 0.8599 for the Logistic Regression baseline. At the chosen threshold it finds 57% of future transactions with 55% precision. The final pipeline is saved, tested, and served through a FastAPI endpoint, ready for the bank to score new customers.

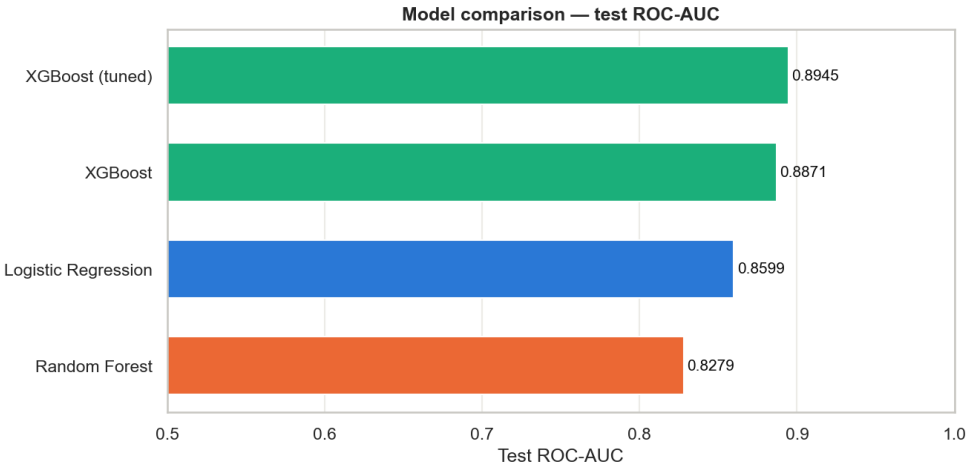
18. Model Comparison Report

Performance of every model on the same test set (precision, recall and F1 at threshold 0.5), with 5-fold cross-validated ROC-AUC.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
XGBoost (tuned)	0.8541	0.3854	0.7595	0.5113	0.8945	0.8941
XGBoost	0.8425	0.3637	0.7565	0.4912	0.8871	0.8877
Logistic Regression	0.7834	0.2865	0.7754	0.4184	0.8599	0.859
Random Forest	0.9071	0.6289	0.1851	0.286	0.8279	0.8324

Recommended for production: XGBoost (tuned). It has the best ROC-AUC on the test set (0.8945) and in cross-validation, small variation across folds, native imbalance handling, and a small, fast pipeline whose threshold (0.683) the bank can adjust. Logistic Regression is an interpretable fallback; Random

Forest is not recommended (lowest ROC-AUC, lowest recall, strongest overfitting).



19. Report on Challenges Faced

Challenge	Technique used	Reason
Class imbalance (~10% positives)	Stratified split and CV, class weights, ROC-AUC and F1 instead of accuracy, F1-optimal threshold	Accuracy rewards always predicting 'no transaction'; class weights fix the bias without synthetic rows
Anonymized features	Statistical EDA; model-based feature importance	No feature meanings, so no domain feature engineering
Weak signal per feature	Kept all 200 features; additive models (boosting, LR)	The prediction comes from combining many weak signals
Identifier column (ID_code)	Excluded from the features	Unique per customer; no predictive meaning
Different feature scales	StandardScaler inside the pipeline	Needed for Logistic Regression; refitted per CV fold, no leakage
Outliers (0.07% of cells, 12.4% of rows)	Kept	Natural tails; removing them drops many positives; trees are robust
Large dataset (200k x 200)	hist method, RandomizedSearchCV, parallel training	A full grid would need 27,648 fits
Decision threshold	Chosen on out-of-fold training predictions	Using the test set would inflate the scores
Overfitting (Random Forest train 0.99 vs test 0.83)	Train vs test check, cross-validation, shallow regularised trees	Shallow subsampled trees generalise better on low-signal data